API / Mocks Execution Guide (Init -> Enrich -> Render + MCP)

This guide consolidates the commands and flow for API mock/stub generation in this repo.

- Project root used in examples: `/Users/natarajankanakasabapathy/FECTECH/TestDataGenerator`
- OpenAPI reference used here: `examples/openapi/Sample_complex.yaml`

---

0) Quick Memory Flow

IER = Init -> Enrich (optional) -> Render

- **Init**: import source artifact (OpenAPI/Postman/HAR) into `sdp-mock-v1`
- **Enrich (optional)**: LLM fills missing examples / error envelopes
- **Render**: generate runnable stubs/fixtures/contracts

---

1) One-Time Setup

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry install
```

If using MCP server:

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry install --extras mcp
```

---

2) General Mock Flow (No LLM Enrichment)

Step 1: Init (OpenAPI -> mock config)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-init --from examples/openapi/Sample_complex.yaml --output moc
```

Step 2: Lint

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-lint --config mocks/sample_complex.yaml
```

Step 3: Render (WireMock/JSON/Postman/Pact)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-render --config mocks/sample_complex.yaml --output stubs/samp
```

---

3) LLM-Enhanced Flow (Init -> Enrich -> Render)

Step 0: Export LM Studio provider vars

```
export SDP_LLM_PROVIDER="lm-studio"
export SDP_LLM_BASE_URL="http://localhost:1234/v1"
export SDP_LLM_MODEL="google/gemma-4-e4b"
```

Step 1: Init

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-init --from examples/openapi/Sample_complex.yaml --output moc
```

Step 2: Enrich (optional, recommended)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-enrich --config mocks/sample_complex.yaml --output mocks/samp
```

Step 3: Lint enriched config

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-lint --config mocks/sample_complex.enriched.yaml
```

Step 4: Render from enriched config

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-render --config mocks/sample_complex.enriched.yaml --output s
```

If skipping enrich, render from `mocks/sample_complex.yaml`.

---

4) Run WireMock After Render

After `mock-render`, start WireMock against the rendered root dir:

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
java -jar ~/tools/wiremock.jar --root-dir stubs/sample_complex/wiremock --port 8081 --verbos
```

Quick sanity check (new terminal):

```
curl http://localhost:8081/__admin/mappings | head
```

Notes: - WireMock runs from `stubs/.../wiremock` output, not from data `generate` output. - Use Bruno/Postman against `http://localhost:8081`.

---

5) MCP + LM Studio (Step-by-Step)

Step 1: Confirm MCP settings in LM Studio

In LM Studio:

- `Settings -> Developer -> Model Context Protocol (MCP)`
- Ensure MCP is enabled
- Use **Reveal `mcp.json`** to open the actual file LM Studio reads

Step 2: Find your Poetry venv path (terminal)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry env info --path
```

Example from this machine:

- `/Users/natarajankanakasabapathy/Library/Caches/pypoetry/virtualenvs/testdatageneration-`

Step 3: Configure `mcp.json` (recommended: direct Python path)

```
{
  "mcpServers": {
    "synthetic-data-platform": {
      "command": "/Users/natarajankanakasabapathy/Library/Caches/pypoetry/virtualenvs/testda
      "args": ["-m", "mcp_server.server"],
      "cwd": "/Users/natarajankanakasabapathy/FECTECH/TestDataGenerator",
      "env": {
        "SDP_LLM_PROVIDER": "lm-studio",
        "SDP_LLM_BASE_URL": "http://localhost:1234/v1",
        "SDP_LLM_MODEL": "google/gemma-4-e4b"
      }
    }
  }
}
```

Why direct Python path is preferred: - avoids `poetry`/shell PATH issues in GUI apps - reduces chance of stdio noise that can break MCP JSON-RPC framing

Step 4: Restart LM Studio

After editing `mcp.json`, fully restart LM Studio.

Step 5: Verify connected server

In MCP panel, confirm `synthetic-data-platform` is connected.

Step 6: Test with a simple prompt in LM Studio chat

```
You are connected to the synthetic-data-platform MCP server.

1) Call list_examples and confirm examples/openapi/Sample_complex.yaml exists.
2) Explain in 3 lines the flow: mock-init -> optional mock-enrich -> mock-render.
3) Return only concise actionable steps.
```

---

6) Full MCP Prompt (End-to-End)

Use this prompt in LM Studio chat:

```
Use the synthetic-data-platform tools and do this end-to-end:

- Source: examples/openapi/Sample_complex.yaml
- Output mock config: mocks/sample_complex.yaml
- Enriched config: mocks/sample_complex.enriched.yaml
- Render output dir: stubs/sample_complex
- Formats: wiremock,json,postman,pact
- examples=5, seed=42, match-mode=any
- LLM provider context: lm-studio at http://localhost:1234/v1 with model google/gemma-4-e4b

Tasks:
1) Run mock-init from the OpenAPI source.
2) Run mock-enrich (optional step, but do it now).
3) Run mock-lint on enriched config.
4) Run mock-render with the options above.
5) Return a short checklist of what was generated and the exact WireMock start command for m

If a step fails, continue with best-effort fallback and clearly mark failed step.
```

---

7) Optional `curl` Prompt Test (LM Studio OpenAI-Compatible API)

Use when testing model responses directly via HTTP:

```
curl -s http://localhost:1234/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "google/gemma-4-e4b",
    "temperature": 0.1,
    "messages": [
      {
        "role": "user",
```

```
        "content": "Use the synthetic-data-platform tools and do this end-to-end: source exa
      }
    ]
  }'
```

Note: - Plain `/v1/chat/completions` can return text without actually invoking MCP tools depending on your LM Studio tool-calling setup.

---

8) Troubleshooting (Most Common)

Symptom: `Starting Synthetic Data Platform MCP server (transport=stdio)` and then nothing

This is normal in stdio mode. Server is waiting for client messages.

Symptom: JSON-RPC parse errors (`Invalid JSON`, EOF, blank line)

Likely stdout pollution. Fixes: - use direct Python path in `mcp.json` command - avoid shell wrappers (`zsh -lc ...`) - avoid inline `export ... && ...` in command args - keep env vars inside `env` block

Symptom: LM Studio says server not connected

Check: - correct `cwd` - valid JSON in `mcp.json` - restart LM Studio after edits - `poetry install --extras mcp`

---

9) Fast Copy-Paste Blocks

General (3 commands)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
poetry run python main.py mock-init --from examples/openapi/Sample_complex.yaml --output moc
poetry run python main.py mock-render --config mocks/sample_complex.yaml --output stubs/samp
```

LLM (4 commands + exports)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
export SDP_LLM_PROVIDER="lm-studio"
export SDP_LLM_BASE_URL="http://localhost:1234/v1"
export SDP_LLM_MODEL="google/gemma-4-e4b"
poetry run python main.py mock-init --from examples/openapi/Sample_complex.yaml --output moc
poetry run python main.py mock-enrich --config mocks/sample_complex.yaml --output mocks/samp
poetry run python main.py mock-render --config mocks/sample_complex.enriched.yaml --output s
```

WireMock run

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
java -jar ~/tools/wiremock.jar --root-dir stubs/sample_complex/wiremock --port 8081 --verbos
```